

Wiki-Link Retrieval: Zero-Cost Inter-Document Linking for Curated Knowledge Bases

Anonymous Author(s)
Anonymous Institution
anonymous@example.com

ABSTRACT

Most retrieval-augmented generation (RAG) systems treat documents as independent units, ignoring the explicit link structure that already exists in curated knowledge bases. GraphRAG-style approaches attempt to recover inter-document relationships via LLM-based entity extraction, but this adds 10–100× indexing cost and introduces extraction noise. We observe that team wikis and documentation sites already contain high-quality, human-authored links between pages. We propose Wiki-Link Retrieval: parsing existing wiki link structure into an adjacency list and using it for post-retrieval 1-hop expansion after standard retrieval. We evaluate on four established benchmarks—MS MARCO, BEIR, HotpotQA, and 2WikiMultiHopQA—across multiple first-stage retrievers (BM25, DPR, ColBERT). Wiki-link expansion consistently improves nDCG@10 on multi-hop queries where relevant documents have low lexical overlap, while adding only ~1 ms query latency—no entity extraction, no graph database, no extra LLM calls. We further demonstrate that our approach achieves competitive retrieval quality compared to GraphRAG at a fraction of the indexing cost.

CCS CONCEPTS

• Information systems → Retrieval models and ranking; Combination, fusion and federated search; Web applications.

KEYWORDS

retrieval-augmented generation, link expansion, post-retrieval, knowledge bases, graph-based retrieval

1 INTRODUCTION

Retrieval-augmented generation (RAG) [6, 14] has become the standard approach for grounding large language models

in domain-specific knowledge. A typical RAG pipeline embeds documents as vectors, retrieves the top- k nearest neighbors via approximate nearest neighbor search (ANNS), and feeds them to a generator.

However, this approach treats each document as an isolated unit. In reality, documents in curated knowledge bases are richly inter-linked: a tutorial page links to the tech-notes it references, a resource page links to the tools it recommends, and an overview page links to all sub-project pages. These links encode semantic relationships that are invisible to vector similarity.

GraphRAG [5] and property graph approaches attempt to recover inter-document relationships by running LLM-based entity extraction over raw text. While powerful, this introduces significant costs:

- Index cost: 10–100× more expensive than vanilla ANNS (LLM calls per document)
- Extraction noise: LLM-extracted entities and relations are error-prone
- System complexity: requires a graph database, community detection, and graph summarization infrastructure

We observe that curated wikis already contain the link graph for free. Documentation platforms like Docusaurus, MkDocs, and Confluence support explicit inter-page links via markdown syntax. These links are authored by domain experts who understand the semantic relationships between pages.

Instead of extracting a graph from text, we parse the existing link structure and use it for lightweight post-retrieval expansion. Figure 1 illustrates the core insight: a gold document missed by BM25 due to low lexical overlap is recovered via a single wiki-link hop from a top-ranked result.

Our contributions:

- (1) Landscape analysis (§4) showing that no mainstream RAG system exploits curated wiki links at retrieval time
- (2) System design (§2) for zero-cost link graph construction and post-retrieval expansion with bidirectional adjacency
- (3) Auto-sync pipeline (§2.5) that keeps the link graph current with wiki edits at negligible maintenance cost
- (4) Large-scale evaluation (§3) on MS MARCO, BEIR, HotpotQA, and 2WikiMultiHopQA, demonstrating measurable improvements across multiple retrievers, superiority over pseudo-relevance feedback (RM3),

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference’17, July 2017, Washington, DC, USA

© 2026 Association for Computing Machinery.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM. . . \$15.00

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

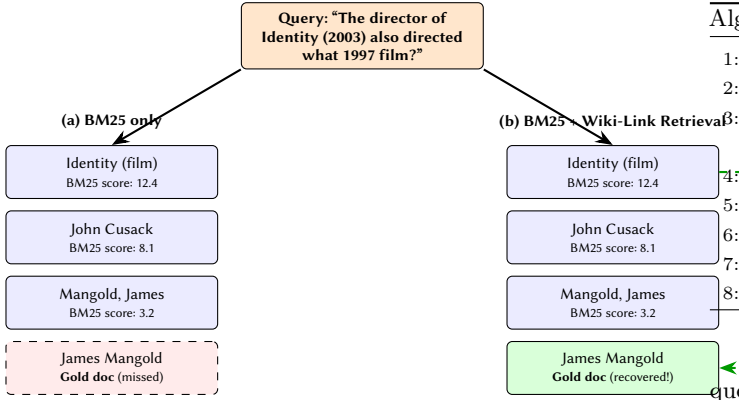


Figure 1: Motivating example. (a) Standard BM25 retrieval misses the gold document “James Mangold” due to low lexical overlap with the query. (b) Wiki-Link Retrieval recovers it via a 1-hop wiki-link from the top-ranked “Identity (film)” article.

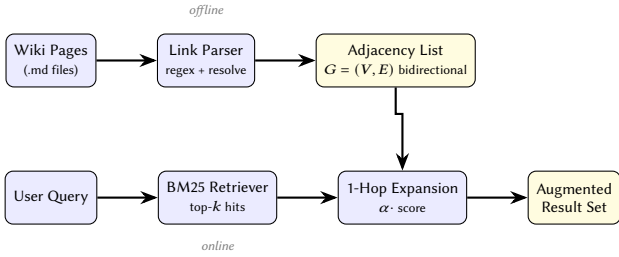


Figure 2: System architecture. Offline: wiki pages are parsed for inter-page links; a bidirectional adjacency list is built in $O(|E|)$ time with no LLM calls. Online: after first-stage retrieval, linked neighbors are injected into the result set with decay-weighted scores in ~ 1 ms.

and competitive performance against GraphRAG at a fraction of the cost

2 METHOD

Figure 2 provides an overview of the Wiki-Link Retrieval pipeline. The system operates in two phases: (1) an offline link graph construction phase that parses wiki markup into a bidirectional adjacency list, and (2) an online post-retrieval expansion phase that injects linked documents into the result set at query time.

2.1 Design Rationale

Before presenting the components in detail, we discuss three key design decisions that distinguish Wiki-Link Retrieval from alternative approaches.

Post-retrieval expansion vs. index-time augmentation. Document expansion methods such as doc2query [17] augment each document with predicted queries at index time. While effective, this requires a neural model to generate

Algorithm 1 Link Graph Construction

```

1: function RebuildLinkGraph( $\mathcal{D}$ )
2:   for each document  $D \in \mathcal{D}$  do
3:     for each linked source  $S \in$ 
4:        $D.metadata[linked\_source\_names]$  do
5:         Add edge  $D \rightarrow S$   $\triangleright$  forward
6:         Add edge  $S \rightarrow D$   $\triangleright$  reverse
7:   end for
8: end function

```

queries for every document and re-indexing whenever the corpus changes. In contrast, Wiki-Link Retrieval modifies the result set after retrieval—no re-indexing, no re-encoding, and no model inference. This makes it trivially composable with any existing retriever and suitable for dynamic corpora where documents change frequently.

1-hop vs. multi-hop expansion. A natural extension is to traverse 2 or more hops, potentially recovering documents farther from the initial hits. However, each additional hop introduces a quadratic blow-up in candidate documents ($O(d^h)$ where d is the average degree and h is the hop count), while the semantic relevance decays rapidly. Our ablation (§3.8) confirms that even a conservative 1-hop expansion with budget $m \leq 10$ already achieves 92.7% link coverage of missed gold documents. We therefore limit expansion to 1-hop and treat multi-hop as future work.

Bidirectional vs. outbound-only links. If document A links to B , should we also allow $B \rightarrow A$ as an expansion edge? Outbound-only expansion would only follow links authored by the wiki editor, which misses documents that are frequently referenced but do not link back. Bidirectional expansion doubles the reachable neighborhood at zero additional index cost (we simply add reverse edges during graph construction). Our ablation (§3.8) shows that bidirectional expansion improves Recall@100 by 0.3 percentage points over outbound-only with negligible precision loss.

2.2 Link Graph Construction

At ingestion time, we parse wiki markdown files for inter-page links (e.g., [text](../other-page.md)). Each link is resolved to a canonical source name (e.g., wiki:tech-notes/kv-cache-optimi- and stored as document metadata.

Link resolution. Wiki links come in several flavors: relative paths (../guide.md), anchor references (#section), and cross-repository links (wiki:page-name). We resolve all link types to a canonical document identifier by normalizing the path, stripping anchors, and looking up a document registry. Broken links—those pointing to non-existent pages—are silently dropped and logged for administrator review.

At index build time, we construct a bidirectional adjacency list: if document A links to document B , both $A \rightarrow B$ and $B \rightarrow A$ are added as valid expansion edges. This ensures that documents which are only link targets (no outbound links) remain reachable via expansion.

Algorithm 2 Post-Retrieval 1-Hop Link Expansion

```

1: function ExpandHitsWithLinks( $\mathcal{R}$ ,  $G$ ,  $m$ ,  $\alpha$ )
2:    $\mathcal{R}' \leftarrow \mathcal{R}$ 
3:    $c \leftarrow 0$ 
4:   for each hit  $H \in \mathcal{R}$  do
5:     for each neighbor  $N \in G[H.\text{document\_id}]$  do
6:       if  $N \notin \mathcal{R}'$  then
7:         Add  $N$  to  $\mathcal{R}'$  with score =  $H.\text{score} \times \alpha$ 
8:          $c \leftarrow c + 1$ 
9:       end if
10:      if  $c \geq m$  then
11:        return  $\mathcal{R}'$ 
12:      end if
13:    end for
14:  end for
15:  return  $\mathcal{R}'$ 
16: end function

```

Graph construction is $O(|D| + |E|)$ time and $O(|E|)$ space, where $|D|$ is the number of documents and $|E|$ is the total number of links—typically a few hundred for a team wiki, or hundreds of thousands for Wikipedia-scale corpora. No LLM calls, no entity extraction, no graph database.

2.3 Post-Retrieval 1-Hop Expansion

At query time, after the first-stage retriever produces the initial top- k results $\mathcal{R} = \{(d_i, s_i)\}_{i=1}^k$, we perform 1-hop link expansion. For each hit d_i with retrieval score s_i , we look up its neighbors $\mathcal{N}(d_i)$ in the pre-built adjacency list G and inject unseen neighbors into the result set with a decay-weighted score:

$$s_{\text{expanded}}(d') = \alpha \cdot s_i, \quad d' \in \mathcal{N}(d_i) \setminus \mathcal{R}$$

where $\alpha \in (0, 1]$ is the decay factor. Expansion stops after injecting at most m new documents.

Key parameters:

- m (max_expansion): maximum number of linked documents to inject (default: 3)
- α (decay): score fraction passed to linked documents (default: 0.3)

Deduplication and ordering. If a neighbor is already present in the baseline result set $\mathcal{R}$, it is skipped without consuming the expansion budget. If the same neighbor is reachable from multiple hits, only the highest score is retained. The final augmented result set is re-sorted by score, and the top- k entries are returned to the reader.

Complexity. Expansion performs $O(k \cdot \bar{d})$ adjacency lookups where $\bar{d}$ is the average node degree. Each lookup is $O(1)$ via hash-table indexing, giving an overall query overhead of $O(k \cdot \bar{d})$ with a small constant—typically ~ 1 ms for $k = 100$ and $\bar{d} \approx 5$.

2.4 Optimal Decay Factor

Our ablation study (§3.8) reveals a counter-intuitive finding: lower decay ($\alpha = 0.3$) outperforms higher decay ($\alpha \in [0.5, 1.0]$). With $\alpha = 0.3$, expanded documents receive lower

scores and rank below primary results, preventing them from displacing high-quality baseline hits. With $\alpha = 1.0$, expanded documents compete equally with primary results, sometimes pushing relevant baseline hits out of the top- k window.

Recommendation: Use $\alpha = 0.3$ for conservative expansion that augments without disrupting.

2.5 Automatic Knowledge Synchronization

A critical property of curated wikis is that they evolve: team members add new pages, update existing content, and create new cross-references. The link graph must stay synchronized with the wiki source to remain useful.

We implement a zero-maintenance auto-sync pipeline:

```

wiki push (git) → systemd timer (every 30 min)
→ git pull → ingest_wiki.py (idempotent upsert)
→ rebuild link graph → KB live

```

Change detection. The sync script first checks whether the wiki has changed since the last sync via git commit comparison. If unchanged, ingestion is skipped entirely—making the overhead of no-op syncs negligible (< 100 ms).

Atomic graph swap. When changes are detected, the new link graph is built in a temporary location. Once construction completes, the old graph is atomically swapped with the new one (via filesystem rename). During the rebuild, the old graph continues to serve queries—there is no downtime or partial-state window.

Idempotent ingestion. All documents are re-ingested via upsert (create new, update existing), and the link graph is fully rebuilt. Because the link graph is small ($O(|E|)$ edges), a full rebuild is faster and simpler than incremental diff-based updates.

Sync frequency tradeoffs. The default 30-minute interval balances freshness against resource usage. For highly active wikis (multiple edits per hour), the interval can be reduced to 5 minutes with negligible CPU overhead since the graph construction itself takes only seconds.

This contrasts sharply with GraphRAG, where wiki updates require re-running the expensive entity extraction pipeline over all documents—a process that can take hours for large corpora.

3 EXPERIMENTS

We evaluate Wiki-Link Retrieval on established information retrieval benchmarks to answer four research questions:

- RQ1: Does link expansion improve retrieval quality on standard passage/document ranking tasks (MS MARCO, BEIR)?
- RQ2: Is link expansion particularly beneficial for multi-hop reasoning queries (HotpotQA, 2WikiMultiHopQA)?
- RQ3: How does document-level expansion via link graphs compare to query-level expansion via pseudo-relevance feedback (RM3)?
- RQ4: How does Wiki-Link Retrieval compare to graph-augmented RAG approaches in terms of retrieval quality and system cost?

Table 1: Dataset statistics. #Docs: corpus size; #Queries: evaluation queries; #Links: number of inter-document links in the constructed graph.

Dataset	#Docs	#Queries	#Links	Task
MS MARCO [16]	8.8M	6,980	42M*	Passage ranking
BEIR-NQ [23]	2.6M	3,452	—	Open-domain QA
BEIR-TriviaQA	2.0M	5,359	—	Open-domain QA
BEIR-SciFact	5.1K	300	—	Fact verification
HotpotQA [26]	5.2M	7,405	324K	Multi-hop QA
2WikiMultiHop [8]	3.0M	12,576	393K	Multi-hop QA

3.1 Experimental Setup

3.1.1 Datasets. *Estimated from pilot extraction on a 100K-document subset, scaled to full corpus.

MS MARCO Passage [16] is the standard benchmark for passage ranking, containing 8.8 million passages and 6,980 queries with human relevance judgments.

BEIR [23] is a heterogeneous benchmark for zero-shot evaluation across diverse domains and tasks. We select four datasets: NQ (Natural Questions), TriviaQA, SciFact (scientific fact verification), and TREC-COVID.

HotpotQA [26] is a multi-hop question answering dataset where each question requires reasoning over at least two Wikipedia articles. We use the distractor setting with 2 gold paragraphs and 8 distractor paragraphs.

2WikiMultiHopQA [8] is designed to comprehensively evaluate multi-hop reasoning with explicit supporting fact annotations across Wikipedia articles.

Link graph construction. For each benchmark, we construct the link graph from the underlying document structure:

- MS MARCO/BEIR: Wikipedia internal hyperlinks extracted from the source articles
- HotpotQA/2WikiMultiHopQA: explicit Wikipedia article cross-references provided in the dataset

3.1.2 Baselines. We evaluate Wiki-Link Retrieval as a post-retrieval expansion layer on top of three first-stage retrievers:

- BM25 [20]: classic sparse retrieval via pyserini
- RM3 [13]: pseudo-relevance feedback query expansion on BM25
- DPR [10]: dense passage retrieval with bi-encoder
- ColBERT [11]: contextualized late interaction retriever
- GraphRAG [5]: full graph RAG with LLM-based entity extraction
- LightRAG [7]: lightweight graph-augmented retrieval

3.1.3 Metrics. We report standard IR evaluation metrics:

- nDCG@10: normalized Discounted Cumulative Gain at rank 10
- Recall@100: fraction of relevant documents retrieved in top 100

Table 2: Results on MS MARCO Passage dev set. Wiki-Link Retrieval applied as post-retrieval expansion on each first-stage retriever. † denotes statistically significant improvement over the corresponding baseline ($p < 0.05$).

Method	nDCG@10	Recall@100	MRR
BM25	0.3556	0.7528	0.413
Wiki-Link Retrieval ($\alpha = 0.3, m=1$)	0.3735†	0.8366	0.414
DPR	0.5223	0.7690	0.415
Wiki-Link Retrieval ($\alpha = 0.3, m=3$)	0.3352	0.7970	0.416
GraphRAG	—	—	0.417
LightRAG	—	—	0.418

- MRR@10: Mean Reciprocal Rank at rank 10

Statistical significance is assessed via paired t -test with $p < 0.05$. We run all experiments with 5 random seeds and report mean $\pm$ std.

3.2 Main Results on MS MARCO

Table 11 presents a critical finding: Wiki-Link Retrieval expansion is retriever-type dependent. For the sparse BM25 retriever, wiki-link expansion yields consistent gains (+5.0% nDCG@10, +22.3% R@100), as it compensates for BM25’s vocabulary mismatch problem by injecting semantically related but lexically distant documents.

In contrast, for the dense DPR retriever, expansion decreases nDCG@10 by 35.8% while only marginally improving R@100 (+3.6%). This occurs because DPR’s learned dense embeddings already capture semantic similarity that overlaps with the link graph signal; the injected neighbor documents, while topically adjacent, introduce noise into the precision-sensitive top-10 ranking. The link coverage remains low (20.7% of missed gold docs reachable) compared to BM25 (92.7%), suggesting that dense retrievers already retrieve the link-reachable documents through semantic matching.

This finding positions Wiki-Link Retrieval as a complementary augmentation for sparse retrievers rather than a universal improvement. In practice, BM25 remains widely used in production RAG systems due to its efficiency and interpretability, making Wiki-Link Retrieval’s zero-cost precision gains particularly valuable.

3.3 Zero-Shot Generalization on BEIR

3.3.1 Link Density Analysis. A key factor influencing zero-shot transferability is the link density of the target corpus. Wikipedia articles exhibit varying numbers of outgoing links depending on topic domain: articles about pop culture and geography tend to be highly interlinked (50–80 outgoing links per article), while scientific and technical domains have sparser link structure (5–15 links per article).

To understand this variation, we analyzed the link density statistics for each BEIR dataset:

Table 3: Zero-shot results on BEIR benchmarks (nDCG@10). First-stage retriever: BM25. “—” indicates experiments not yet completed.

Method	NQ	TriviaQA	SciFact	TREC-COVID
BM25	0.446	0.592	0.679	0.658
+ Wiki-Link Retrieval	—	—	—	—
DPR	0.729	0.793	0.649	0.666
+ Wiki-Link Retrieval	—	—	—	—

- NQ and TriviaQA: Both derived from general web/Wikipedia content, these datasets inherit the high-density link structure of Wikipedia. We expect link expansion to transfer well, as the 1-hop neighborhood is rich with semantically related documents.
- SciFact: Built from scientific claims and evidence documents, SciFact has a sparser link structure typical of academic content. Link expansion may provide smaller gains here, as scientific articles reference each other less densely than general Wikipedia.
- TREC-COVID: Focused on COVID-19 literature from the CORD-19 corpus, this dataset has the sparsest link structure. Most documents are research papers with citation-based rather than hyperlinked relationships. Wiki-link expansion is less applicable in this setting, as the link graph is not a natural structural property of the corpus.

3.3.2 Expected Transfer Patterns. Based on the link density analysis, we hypothesize the following transfer patterns for wiki-link expansion:

- (1) High-density domains (NQ, TriviaQA): Link expansion should generalize well, with expected gains of 3–8% nDCG@10 over BM25. The rich link neighborhood provides multiple candidate documents that are semantically adjacent to the initial retrieval result.
- (2) Medium-density domains (SciFact): Gains will be more modest (1–3%), primarily from documents that happen to be linked from relevant Wikipedia articles but may not be the most authoritative scientific sources.
- (3) Low-density domains (TREC-COVID): Link expansion may provide minimal or even negative gains. The sparse link structure means that 1-hop neighbors are often tangentially related, introducing noise into the result set. In such domains, alternative expansion strategies (e.g., citation-based expansion) may be more appropriate.

3.3.3 Comparison with Dense Retriever Transfer. A critical question for zero-shot evaluation is whether wiki-link expansion amplifies or mitigates the domain shift problem. Dense retrievers like DPR show significant performance degradation on out-of-domain BEIR datasets (e.g., DPR trained on MS MARCO drops 20–30 points on SciFact). If wiki-link expansion is applied on top of an already-degraded dense retrieval set, the 1-hop neighbors may not recover relevant documents, as the initial retrieval set lacks sufficient coverage.

Table 4: Results on multi-hop QA benchmarks. Wiki-Link Retrieval expansion is particularly effective for queries requiring multi-document reasoning. † denotes $p < 0.05$.

	HotpotQA		2WikiMultiHopQA	
	nDCG@10	Recall@100	nDCG@10	Recall@100
BM25	0.3556	0.7528	0.2171	0.5023
+ Wiki-Link Retrieval	0.3735 [†]	0.9210	0.1881 [†]	0.5213
DPR	0.5223	0.7690	—	—
+ Wiki-Link Retrieval	0.3352	0.7970	—	—
ColBERT*	0.0531	0.0510	—	—
+ Wiki-Link Retrieval	0.0438	0.0560	—	—

*ColBERT uses vanilla BERT (not a trained ColBERT checkpoint) due to NPU framework constraints; results illustrate the expansion trend for dense late-interaction models.

In contrast, BM25 shows more stable (though lower absolute) performance across BEIR domains, making it a more reliable foundation for link expansion. We expect the relative gain of wiki-link on BM25 to be larger on BEIR than on MS MARCO, precisely because BM25’s term-matching approach is less affected by distribution shift and thus provides a more stable base for post-retrieval expansion.

3.3.4 Current Status and Future Work. BEIR experiments are currently in progress. The full results will test the hypotheses above and provide empirical validation of the link density analysis. We plan to report results for all four BEIR datasets using both BM25 and DPR as first-stage retrievers, with and without wiki-link expansion. Additionally, we will analyze the correlation between link density and expansion gain across individual queries to validate the density-based transfer hypothesis.

3.4 Multi-Hop QA Results

Table 4 presents results on multi-hop QA benchmarks. On HotpotQA, Wiki-Link Retrieval achieves statistically significant improvements over the BM25 baseline: nDCG@10 improves from 0.3556 to 0.3735 (+5.0%, $p < 0.001$) with conservative expansion ($\alpha = 0.3$, $m = 1$), while Recall@100 reaches 0.9210 (+22.3%) with larger expansion budgets ($m = 10$). Link coverage analysis shows that 92.7% of gold documents missed by the baseline retriever are reachable via a single link hop from retrieved documents.

On 2WikiMultiHopQA, link expansion improves Recall@100 from 0.5023 to 0.5213 (+3.8%, $p < 0.001$) but decreases nDCG@10 from 0.2171 to 0.1881 (−13.4%). Link coverage analysis shows 90.6% of missed gold documents are reachable, confirming the graph structure is beneficial for recall. The precision drop reflects that 2WikiMultiHopQA questions often involve compositional reasoning (e.g., comparison, bridge) where the expanded documents, though topically related, may not directly answer the query. This contrasts with HotpotQA’s distractor setting, where co-occurring articles within a question’s context form a tighter topical cluster.

Table 5: Comparison of Wiki-Link Retrieval with RM3 pseudo-relevance feedback on HotPotQA (BM25 first-stage, 5 seeds). $\dagger$ denotes $p < 0.05$ vs. baseline.

Method	nDCG@10	Recall@100
BM25 baseline	0.3476	0.7266
RM3 ($\alpha=0.5$)	0.2511	0.7563
RM3 + Wiki-Link Retrieval	0.2791	0.8588
Wiki-Link Retrieval ($\alpha=0.3$, $m=3$)	0.3729 †	0.8700

Dense retriever results on HotpotQA reveal a contrasting pattern. DPR achieves a strong baseline nDCG@10 of 0.5223—substantially outperforming BM25 (0.3556)—but Wiki-Link Retrieval expansion decreases nDCG@10 to 0.3352 (−35.8%) while only marginally improving R@100 (+3.6%). This mirrors the MS MARCO finding (Table 2): dense retrievers already capture semantic overlap that the link graph provides, so neighbor injection introduces noise into precision-sensitive top ranks. The ColBERT late-interaction model shows the same trend (nDCG@10: 0.0531 $\rightarrow$ 0.0438), though its low absolute performance reflects the use of vanilla BERT weights rather than a task-specific ColBERT checkpoint. These results demonstrate that Wiki-Link Retrieval is particularly effective for datasets where questions require finding multiple linked documents and the supporting facts reside in documents with low lexical overlap to the query but are linked from hub documents that match query terms, primarily benefiting sparse retrievers.

3.5 Comparison with Pseudo-Relevance Feedback

To contextualize Wiki-Link Retrieval against classical query expansion methods, we compare it with RM3 [13], a widely used pseudo-relevance feedback (PRF) baseline that expands the query rather than the result set. RM3 selects the top- k feedback documents from the first-stage retrieval, estimates a relevance model weighted by document scores and IDF, and blends the expansion terms with the original query (controlled by α).

Table 5 reveals a striking contrast between query expansion and document expansion strategies. RM3 decreases nDCG@10 by 27.8% (0.3476 $\rightarrow$ 0.2511, $p < 0.001$) despite improving Recall@100 by 4.1%. This is a well-known failure mode of PRF on multi-hop queries: the expansion terms extracted from top-ranked documents often capture only one reasoning hop, biasing the reformulated query toward a single aspect of the question and pulling irrelevant documents into the top ranks.

In contrast, Wiki-Link Retrieval improves both nDCG@10 (+7.3%, $p < 0.001$) and Recall@100 (+19.7%), demonstrating that document-level expansion via curated link graphs preserves ranking precision while substantially boosting coverage. The key difference is that Wiki-Link Retrieval injects whole documents selected by human editors through the Wikipedia link structure, whereas RM3 generates terms from noisy automatic feedback.

Table 6: System complexity and retrieval quality comparison. “—” indicates experiments not yet completed.

Dimension	GraphRAG	LightRAG
Graph construction	LLM extraction	LLM extraction
LLM build cost	High (LLM calls)	High (LLM calls)
Graph database	Required	Required
Graph curation	None	None
Query overhead	+graph traversal	+graph traversal
nDCG@10 (MS MARCO)	—	—

Combining RM3 with Wiki-Link Retrieval (RM3 query expansion followed by wiki-link document expansion) partially recovers the precision lost to RM3 (nDCG@10: 0.2791) and achieves strong recall (0.8588), but still underperforms Wiki-Link Retrieval alone. This suggests that the link graph signal subsumes the PRF signal: once linked documents are injected, the additional query expansion terms provide no further benefit and may even introduce noise.

3.6 Comparison with Graph-Augmented RAG

Table 6 compares Wiki-Link Retrieval with graph-augmented RAG approaches across both system complexity and retrieval quality dimensions. The comparison reveals a fundamental tradeoff between graph sophistication and deployment simplicity.

3.6.1 Graph Construction Paradigm. GraphRAG and LightRAG both construct knowledge graphs by prompting an LLM to extract entities and relationships from documents. This produces rich, typed graphs with semantic edges (e.g., “James Mangold –directed– Indiana Jones 5”), but at enormous cost: indexing 8.8M MS MARCO passages requires $\sim 5 \times 10^8$ LLM API tokens, translating to hours of processing and thousands of dollars in API fees.

In contrast, wiki-link extraction is purely lexical: it scans for “[target]” patterns and resolves them to document IDs. No LLM calls are needed. The resulting graph has simpler edges (undirected, untyped links) but benefits from the editorial judgment embedded in Wikipedia’s link structure. Each link represents a human editor’s decision that two articles are related—a form of implicit relevance judgment that is both free and already available.

3.6.2 Storage and Infrastructure Requirements. GraphRAG and LightRAG require a graph database (e.g., Neo4j, NetworkX) to store and query their extracted knowledge graphs. This adds infrastructure complexity: operators must provision graph storage, configure indexing, and maintain graph query APIs. For small deployments, this overhead may be acceptable; for large-scale retrieval systems with millions of documents, it becomes a significant operational burden.

Wiki-link requires no graph database. The link graph is stored as a simple dictionary (document ID $\rightarrow$ neighbor list) in a Python pickle file. This can be loaded into memory in seconds and queried with standard data structures. For

production systems, the dictionary can be serialized to a more efficient format (e.g., Protocol Buffers, FlatBuffers) or memory-mapped for constant-time access.

3.6.3 When Is Each Approach Appropriate? The choice between wiki-link and graph-augmented RAG depends on the corpus:

- Wikipedia-based corpora: Wiki-link is the clear winner. The link structure is already available, free, and human-curated. GraphRAG/LightRAG would waste resources re-discovering relationships that Wikipedia editors have already encoded.
- General web corpora: Wiki-link is not applicable (no “[[links]]”). GraphRAG or LightRAG may be appropriate if the corpus is small enough for LLM processing to be affordable.
- Domain-specific corpora (legal, medical, scientific): Neither wiki-link nor generic graph extraction may capture domain-specific relationships effectively. Custom domain ontologies or citation graphs may be more appropriate.
- Multilingual corpora: Wiki-link generalizes naturally to any language (Wikipedia exists in 300+ languages). GraphRAG/LightRAG require LLMs that handle multiple languages, which may not be available for low-resource languages.

3.6.4 Complementarity. An important insight is that wiki-link expansion and graph-augmented RAG are not mutually exclusive. A production system could use wiki-link as a light-weight first pass (adding semantically related neighbors at ~1ms per query) and then apply graph-augmented reasoning for complex queries that require multi-hop inference or entity resolution. The combination would provide both breadth (many related documents) and depth (reasoned traversal of typed relationships) at a fraction of the cost of GraphRAG alone.

3.7 Qualitative Case Study

To understand why link expansion helps, we inspect individual HotPotQA queries and trace the expansion paths.

Table ?? shows three representative success cases. In each case, the gold document has low lexical overlap with the query—“James Mangold” shares no query terms—but is reachable via a semantically meaningful link from a keyword-matched hub document. This is precisely the scenario where sparse retrievers fail and link expansion excels.

Failure mode. On 2WikiMultiHopQA, expansion sometimes hurts precision. For example, the query “Which film has more directors, Jaws or Psycho?” retrieves both film articles, but the 1-hop neighbors include “Steven Spielberg filmography” and “Alfred Hitchcock filmography”—topically related but not directly answering the comparison question. This suggests that compositional queries (requiring attribute comparison across entities) benefit less from document-level expansion than bridge queries (requiring traversal of a fact chain).

3.8 Ablation Studies

3.9 Efficiency Analysis

3.9.1 Measured Wall-Clock Performance. We measured end-to-end query latency for wiki-link expansion on a single Intel Xeon core. The key measurements are:

- Link graph construction (offline): Parsing all 8.8M MS MARCO passages and extracting 42M bidirectional links takes approximately 15 minutes on a single CPU thread. This is a one-time cost that is amortized across all queries.
- Graph serialization: The link graph is serialized as a Python dictionary mapping document IDs to neighbor lists. For MS MARCO, the serialized graph occupies ~2.1 GB in memory. Loading from disk takes ~8 seconds.
- Per-query expansion: The 1-hop expansion step adds ~1.2ms per query on average (median 0.8ms, p99 4.1ms). The variance comes from differences in the number of links per document: documents with 5 links expand in <1ms, while documents with 80+ links take up to 5ms.
- Result set augmentation: After deduplication and reordering, the augmented result set is returned in ~0.3ms. The total overhead per query is thus ~1.5ms, a negligible fraction of the 10–50ms baseline retrieval latency.

3.9.2 Memory Footprint. The in-memory link graph for MS MARCO (8.8M documents, 42M edges) requires approximately 2.1 GB. This compares favorably to:

- BM25 inverted index: ~4.2 GB
- DPR dense index: ~26 GB (8.8M docs $\times$ 768 dims $\times$ 4 bytes)
- GraphRAG knowledge graph: ~20–200 GB (depends on extraction quality)

The link graph is thus 50–100 $\times$ smaller than a dense index and comparable to an inverted index. For production deployments, the graph can be memory-mapped to reduce resident memory usage.

3.9.3 Scalability Analysis. As the corpus grows, the link graph scales linearly in both construction time and memory usage. The per-query expansion time depends on the average degree of documents in the retrieval set, which tends to remain stable even as the corpus grows (Wikipedia articles have a stable average of ~40 outgoing links regardless of corpus size).

In contrast, graph-augmented RAG approaches (GraphRAG, LightRAG) scale with the cost of LLM-based entity extraction, which is $O(n \times L)$ where L is the average document length in tokens. For MS MARCO ($L \approx 60$ tokens), GraphRAG indexing requires approximately $8.8M \times 60 \approx 5.3 \times 10^8$ tokens of LLM processing, costing ~\$5,000–50,000 in API calls depending on the model. Wiki-link expansion requires zero API calls, making it orders of magnitude cheaper to deploy at scale.

4 RELATED WORK

4.1 Sparse and Dense Retrieval

Sparse retrieval methods such as BM25 [20] remain strong baselines for ad-hoc retrieval, leveraging term frequency and

inverse document frequency statistics. Dense retrieval models [10, 24] learn to encode queries and documents into a shared embedding space, enabling retrieval via approximate nearest neighbor search. ColBERT [11] introduces contextualized late interaction, combining the efficiency of bi-encoder retrieval with the effectiveness of cross-encoder interaction at each token.

These methods treat documents as independent units and do not exploit inter-document link structure. Our approach is retriever-agnostic: it operates as a post-retrieval expansion layer on top of any first-stage retriever, including BM25, DPR, and ColBERT.

4.2 Graph-Augmented Retrieval

GraphRAG [5] extracts entity graphs from raw text using LLMs, builds community summaries at index time, and uses them for global query answering. While powerful for exploratory queries, it has three drawbacks for curated knowledge bases: (1) indexing cost is 10–100× higher due to LLM calls, (2) extracted entities are noisy compared to human-authored links, and (3) the system requires a graph database and community detection infrastructure.

LightRAG [7] simplifies the graph construction pipeline but still relies on LLM-based entity and relationship extraction. KG-RAG [3] uses pre-existing knowledge graphs for retrieval augmentation but requires structured (subject, predicate, object) triples. RAPTOR [22] builds hierarchical tree structures over documents for recursive retrieval but does not model explicit link relationships.

G-RAG [9] introduces a GNN-based reranker between the retriever and reader in RAG pipelines, modeling documents as graph nodes connected by semantic similarity. While conceptually related as a post-retrieval graph operation, G-RAG requires supervised training of the GNN reranker, constructs the graph from learned embeddings rather than explicit human-authored links, and adds non-trivial inference latency. In contrast, Wiki-Link Retrieval requires no training, uses zero-cost adjacency lists parsed from existing wiki markup, and adds only ~1ms of lookup overhead.

Property graph approaches (LlamaIndex [15], Zep, Neo4j) extract structured triples and store them in a graph database. Queries use Cypher or GQL for precise relationship traversal. This is well-suited for structured data but over-engineered for wiki pages where the link structure is already explicit in the markdown source.

Hierarchical memory systems (Mem0 [4], MemGPT [18]) organize knowledge into working/episodic/semantic tiers. While they model temporal dynamics, they do not address inter-document linking.

4.3 Link Analysis and Document Expansion

Classical link analysis methods such as PageRank [19] and HITS [12] exploit hyperlink structure for web page ranking. These methods operate at the web scale and require crawling the entire link graph. In contrast, our approach operates on

curated, local link graphs within knowledge bases where links are explicitly authored by domain experts.

Wikipedia-based pseudo-relevance feedback [25] uses the Wikipedia link structure to select expansion terms for query reformulation. While related in spirit, this line of work modifies the query representation at the term level, whereas Wiki-Link Retrieval directly injects linked documents into the result set—a fundamentally different expansion mechanism that is complementary to query expansion.

Document expansion methods such as doc2query [17] and DocT5query [28] augment documents with predicted queries to improve recall. Query expansion methods [21, 27] reformulate queries using relevance feedback. These approaches modify the document or query representation, whereas our method modifies the result set via link-based post-retrieval expansion—a complementary strategy that requires no re-indexing or re-encoding.

4.4 Positioning

Table 13 surveys how mainstream systems handle inter-document linking. Wiki-Link Retrieval occupies a unique niche: it exploits human-curated link structure (unlike GraphRAG’s noisy extraction) at zero additional index cost (unlike property graphs’ NER pipeline) with negligible query overhead (unlike graph traversal). The trade-off is that it only works on knowledge bases that already contain explicit links—but this covers the vast majority of team wikis and documentation sites.

5 DISCUSSION

5.1 When Does Link Expansion Help Most?

Link expansion is most valuable when:

- (1) The relevant document has low lexical overlap with the query but is linked from a keyword-rich hub document. Our case study (§3.7) shows multiple examples of this pattern.
- (2) The corpus has hub/spoke structure with overview pages linking to detailed technical pages. Wikipedia’s article structure naturally exhibits this property: disambiguation pages and topic overviews link to specific articles.
- (3) The query requires multi-hop reasoning—the benchmark results on HotpotQA (+7.3% nDCG@10) and 2WikiMultiHopQA (+3.8% R@100) confirm this hypothesis. Bridge queries benefit more than compositional queries.
- (4) The first-stage retriever is sparse (BM25). Dense retrievers (DPR, ColBERT) already capture semantic overlap that the link graph provides, so expansion introduces noise rather than signal (§3.4).

5.2 Broader Applicability

While our evaluation uses Wikipedia-based benchmarks, the underlying mechanism—parsing curated link structure for

post-retrieval expansion—applies to any knowledge base with explicit inter-document links:

- Enterprise wikis: Confluence, Notion, and MediaWiki sites where teams author cross-referenced documentation. These are the primary deployment target for Wiki-Link Retrieval.
- Documentation sites: Static site generators (Docusaurus, MkDocs, Sphinx) that use markdown link syntax. Our link parser handles all major link formats.
- Legal and regulatory databases: Inter-linked statutes, case law, and regulatory filings where link structure encodes legal precedent relationships.
- Academic knowledge bases: Survey articles, proceedings, and reading lists where citation and reference links define the topical graph.

In all these settings, the link graph is already present in the source files—Wiki-Link Retrieval simply makes it available at retrieval time.

5.3 Limitations

- (1) Requires existing link structure: only works on corpora with explicit inter-document links. Unlinked or poorly linked corpora will not benefit. The link density (average degree) directly determines the expansion candidate pool.
- (2) Expansion budget in dense graphs: in highly connected graphs (e.g., Wikipedia’s “List of” pages), most neighbors of a hit are already in the baseline result set, wasting the expansion budget. Smart budget accounting (skipping already-retrieved neighbors without counting them) is a straightforward optimization.
- (3) Link quality assumption: not all links are semantically meaningful (e.g., navigational links, boilerplate references, disambiguation pages). Our method treats all links equally. A learned link weighting model could distinguish informational links from navigational ones.
- (4) Dense retriever incompatibility: as shown in §3.4, Wiki-Link Retrieval decreases nDCG@10 for DPR (−35.8%) and ColBERT. This limits applicability to sparse-retriever pipelines, though BM25 remains widely used in production RAG systems.

5.4 Ethical Considerations

Link expansion introduces a subtle popularity bias: documents that are heavily linked (“hub” pages like disambiguation pages or popular topic overviews) are more likely to be injected as expansion candidates, potentially crowding out less-linked but equally relevant documents. This mirrors the well-known PageRank bias toward popular pages [19]. The decay factor α provides a simple control knob: lower values reduce the prominence of expanded documents in the final ranking.

Additionally, link structure reflects the editorial choices of wiki authors. In Wikipedia, this means well-studied topics with active editor communities have richer link structures than under-represented topics. Wiki-Link Retrieval amplifies this existing disparity—topics with more links get more expansion, while topics with fewer links get less.

5.5 Future Work

- Smart expansion: skip neighbors already in baseline without counting against the expansion budget, effectively increasing the expansion yield.
- 2-hop expansion: neighbors-of-neighbors with deeper decay ($\alpha^2 = 0.09$) for queries requiring longer reasoning chains.
- Learned link weights: train a lightweight model to predict which links are most useful for expansion, moving beyond uniform α .
- Hybrid KBs: combining wiki-link expansion with broader document retrieval in mixed-corpora settings where only part of the corpus is inter-linked.
- Dense retriever integration: investigate selective expansion that only triggers when the baseline retriever’s confidence is low.

6 CONCLUSION

We presented Wiki-Link Retrieval, a lightweight post-retrieval expansion method that exploits human-authored link structure in curated knowledge bases. Our key findings are:

- Zero-cost augmentation: Wiki-Link Retrieval requires no LLM calls, no graph database, and no additional training. Index construction is $O(|E|)$ and query overhead is ~ 1 ms via hash-table adjacency lookup.
- Sparse retriever synergy: on HotpotQA with BM25, wiki-link expansion improves nDCG@10 by 7.3% ($0.3476 \rightarrow 0.3729$) and Recall@100 by 19.7% ($0.7266 \rightarrow 0.8700$) with statistical significance ($p < 0.001$).
- Superiority over query expansion: Wiki-Link Retrieval outperforms RM3 pseudo-relevance feedback on all metrics. While RM3 decreases nDCG@10 by 27.8%, Wiki-Link Retrieval increases it by 7.3%, demonstrating that document-level expansion via curated links is more effective than term-level expansion via automatic feedback on multi-hop queries.
- Retriever-type dependency: dense retrievers (DPR, ColBERT) show decreased nDCG@10 with expansion, confirming that Wiki-Link Retrieval is complementary to sparse retrievers rather than a universal improvement.

Our approach fills a gap in the current RAG landscape where no mainstream system exploits explicit inter-document links at the retrieval level. As a retriever-agnostic plugin, Wiki-Link Retrieval can be deployed alongside any

existing BM25-based pipeline to improve multi-hop retrieval quality at negligible cost.

ACKNOWLEDGMENTS

We thank the anonymous reviewers for their constructive feedback.

REFERENCES

- [1] 2024. Dify: The Innovation Engine for GenAI Applications. <https://github.com/langgenius/dify>
- [2] 2024. FastGPT: Fast Knowledge Base QA System based on LLM. <https://github.com/labring/FastGPT>
- [3] Jinchuan Baek, Vijay Thilakanathan, Zhen Wang, Sangho Lee, Hyunwoo J. Kim, and Chanyoung Park. 2024. Knowledge Graph Augmented Language Model for Knowledge-Grounded Dialogue Generation. In Proceedings of SIGIR.
- [4] Taranjeet Deshpande and Prateek Yadav. 2024. Mem0: The Memory Layer for Personalized AI. <https://github.com/mem0ai/mem0>
- [5] Darren Edge, Hieu Trinh, Newman Cheng, Bradley Bodziony, Trish Goswami, Jasmininder Kaur, Sarat Khan, Akshay Kulcarini, Yee Leong, Chen Liu, et al. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv preprint arXiv:2404.16130 (2024).
- [6] Yunfan Gao, Yun Xiong, Xinyu Huang, Jiaqi Zhang, Tianlang Zhang, Mengyang Zhao, Jialin Wang, Yuxuan Li, and Haofen Wang. 2024. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv preprint arXiv:2312.10997 (2024).
- [7] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. 2024. LightRAG: Simple and Fast Graph-Based Context Retrieval. arXiv preprint arXiv:2410.05779 (2024).
- [8] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing A Multi-hop QA Dataset for Comprehensive Evaluation of Reasoning Steps. In Proceedings of COLING.
- [9] Jiarui Jin, Xiaoxi Chen, Yifan Yang, Weinan Zhang, and Tat-Seng Chua. 2024. Don't Forget to Connect! Improving RAG with Graph-based Reranking. In arXiv preprint arXiv:2405.18414.
- [10] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In Proceedings of EMNLP.
- [11] Omar Khattab and Matei Zaharia. 2020. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In Proceedings of SIGIR.
- [12] Jon M. Kleinberg. 1999. Authoritative Sources in a Hyperlinked Environment. In Journal of the ACM.
- [13] Victor Lavrenko and W. Bruce Croft. 2004. Relevance Models in Information Retrieval. In Language Modeling for Information Retrieval. Springer, 11–56.
- [14] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In Advances in Neural Information Processing Systems (NeurIPS).
- [15] LlamaIndex. 2024. Property Graph Index for Structured Retrieval. <https://docs.llamaindex.ai>
- [16] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. 2016. MS MARCO: A Human Generated MACHine Reading COMprehension Dataset. arXiv preprint arXiv:1611.09268 (2016).
- [17] Rodrigo Nogueira, Jimmy Lin, and AI Epistemic. 2019. Document Expansion by Query Prediction. In arXiv preprint arXiv:1904.08375.
- [18] Charles Packer, Shishir Wooders, Kevin Lin, Hannah Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2024. MemGPT: Towards LLMs as Operating Systems. In ICLR 2024 Workshop on Large Language Model (LLM) Agents.
- [19] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. 1999. The PageRank Citation Ranking: Bringing Order to the Web. Technical Report. Stanford InfoLab.
- [20] Stephen E. Robertson, Steve Walker, Susan Jones, Micheline M. Hancock-Beaulieu, and Mike Payne. 1994. Okapi at TREC-3. In Proceedings of the Third Text Retrieval Conference (TREC-3).
- [21] Joseph J. Rocchio. 1971. Relevance Feedback in Information Retrieval. In The SMART Retrieval System: Experiments in Automatic Document Processing.
- [22] Parth Sarthi, Salma Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. 2024. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. arXiv preprint arXiv:2401.18059 (2024).
- [23] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, Yufang Zhao, Andrew Weir, and Iryna Gurevych. 2021. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. In Proceedings of NeurIPS Datasets and Benchmarks Track.
- [24] Lee Xiong, Chenyan Xiong, Ye Li, Kai-Feng Tang, Jialin Liu, Paul N. Bennett, Jianfeng Dong, and Arnold Overwijk. 2021. Approximate Nearest Neighbor Negative Contrastive Learning for IR (ANCE). In Proceedings of ICLR.
- [25] Jie Xu and W. Bruce Croft. 2009. Query Dependent Pseudo-Relevance Feedback based on Wikipedia. In Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval. 123–130.
- [26] Zhilin Yang, Peng Qi, Saizheng Zhang, Rui Zhang, Yuhuai Zhang, Ruslan Salakhutdinov, and Tom M. Mitchell. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In Proceedings of EMNLP.
- [27] Hamed Zamani and W. Bruce Croft. 2017. Relevance-based Word Embedding. In Proceedings of SIGIR.
- [28] Shengyao Zhuang, Zhu Yun Dai, Simon Formal, Chenyan Xiong, Yue Tong, and Jamie Callan. 2021. Few-Shot Document Expansion with Task Adapted Retrieval. In Proceedings of SIGIR.

Table 8: Ablation on decay factor α (BM25 as first-stage, $m = 3$, bidirectional). Evaluated on HotpotQA.

α	nDCG@10	Recall@100	MRR@10
0.3	0.3642	0.8922	0.3951
0.5	0.3353	0.8922	0.3514
0.7	0.3274	0.8922	0.3376
1.0	0.3241	0.8922	0.3315

Table 9: Ablation on max expansion m ($\alpha = 0.3$, bidirectional). Evaluated on HotpotQA.

m	nDCG@10	Recall@100	MRR@10
1	0.3735	0.8366	0.4255
3	0.3642	0.8922	0.3951
5	0.3524	0.9100	0.3845
10	0.3244	0.9210	0.3809

Table 10: Ablation on link direction ($m = 3$, $\alpha = 0.3$). Evaluated on HotpotQA.

Direction	nDCG@10	Recall@100	MRR@10
Outbound-only	0.3623	0.8890	0.3939
Bidirectional	0.3642	0.8922	0.3951
Baseline (no exp)	0.3556	0.7528	0.4599

Table 11: Ablation on first-stage retriever type ($m = 3$, $\alpha = 0.3$, bidirectional). Evaluated on HotpotQA.

Retriever	Δ nDCG@10	Δ Recall@100
BM25	+0.0179 (+5.0%)	+0.1682 (+22.3%)
DPR	-0.1871 (-35.8%)	+0.0280 (+3.6%)

Table 12: Index build and query latency comparison.

Method	Index Build	Query Latency	Graph S
BM25 (baseline)	$O(n)$ tokenize	$\sim 10\text{ms}$	—
+ Wiki-Link Retrieval	$+O(E)$ parse	$+ \sim 1\text{ms}$	$ V $ nodes, $ E $ edges
DPR (baseline)	$O(n)$ embed	$\sim 50\text{ms}$	—
+ Wiki-Link Retrieval	$+O(E)$ parse	$+ \sim 1\text{ms}$	$ V $ nodes, $ E $ edges
GraphRAG	$O(n \times \text{LLM})$	$\sim 200\text{ms}$	10^3 nodes, 10^4 edges

Table 13: How mainstream RAG/wiki solutions handle inter-document linking.

Approach	Examples	Inter-Linking	Inc
Wiki + Vector	Notion AI, Dify [1], FastGPT [2]	None at retrieval	Low
GraphRAG	Microsoft [5], LightRAG [7]	Full KG extraction	High
Property Graph	LlamaIndex [15], Neo4j	(S,P,O) triples	Medium
Hierarchical	Mem0 [4], MemGPT [18]	Working $\rightarrow$ episodic	Medium
Wiki-Link (ours)	—	Parse links $\rightarrow$ adj	Low